

Data Products in DataOS

The working reference for how Data Products should be defined, grounded, and governed inside an organization, and how DataOS implements that standard.

VERSION

0.1

LAST UPDATED

24 April 2026

OWNER

Animesh Kumar

STATUS

Draft



Contents

About This Guide	3
Glossary	4
Position	6

PART ONE · PRINCIPLES

1. Definition	8
2. Design Principles	10
3. Grounding a Data Product	11
4. Anti-patterns to Refuse	16
5. What This Position Costs	17
6. Is This a Data Product? A Decision Rubric	18
7. Data Product Tiers	20
8. Roles and Responsibilities	23

PART TWO · IMPLEMENTATION

1. The Three-Stage Flow	26
2. The Engine: Through-line Across All Stages	27
3. Discovery, Before the Data Product Exists	28
4. Production, Building the Data Product	31
5. Consumption, Accessing the Data Product	35
6. Governance, Lineage, Observability	37
7. Above the Data Products: the Semantic Layer	38
8. Full-stack and Overlay Patterns	41

About This Guide

Two parts. **Part 1** sets the principles: what a Data Product is, how it is grounded, the anti-patterns it refuses, and the roles around it. **Part 2** describes how DataOS operationalizes those principles across Discovery, Production, and Consumption.

Part 1 is platform-neutral; Part 2 is DataOS-specific. The principles hold regardless of where a team builds.

Glossary

Terms used throughout this guide. Defined here once so they can be used consistently in both parts. A reader familiar with the terminology can skip this section; everyone else should read it once before starting Part 1.

CONTRACT TERMS

- **SLA (Service Level Agreement).** The Data Product's published, measurable commitment to its consumers on freshness, availability, completeness, and quality. Machine-readable, so downstream pipelines and AI agents can gate on it programmatically rather than discovering a problem after the fact.
- **Freshness.** How recently the data in a Data Product reflects its source. Can be continuous (streaming), minutes (CDC replication), hours (incremental batch), or days (full refresh). Published as part of the SLA.
- **Lineage.** The traceable path from a source system through every transformation to the Data Product and onward to every consumer. Column-level lineage traces individual fields, not just whole tables; it is what lets auditors, agents, and compliance reviewers answer "*where did this number come from?*" deterministically.
- **Semantic model.** The bounded context of meaning that a Data Product owns: its column definitions, business rules, domain vocabulary, entities, relationships, and metrics as they exist *within the product's own boundary*. Every Data Product has one by construction. The semantic model is local, self-contained, and travels with the product.
- **Semantic layer.** The architectural tier *above* the collection of Data Products, where individual semantic models are registered and composed into a cross-product view. The semantic layer also holds the organizational ontology: typed relationships between concepts across products (e.g. `campaign_spend` *influences* `pipeline_revenue`) that no single product owns. It goes by several names in different communities: *semantic backend*, *ontology layer*, *context graph*, *knowledge graph*. The naming varies; the function is the same. The semantic layer does not own data; Data Products do. It holds the *meaning* of data and the relationships between products. See [Part 2 §7](#).

PERFORMANCE TERMS

- **Latency.** How long a single query or request takes to return a result. Measured in milliseconds for operational products (point lookups, real-time APIs), seconds for analytical products (dashboards, ad-hoc SQL), minutes for batch training workloads. Different consumption patterns demand different latency bounds.
- **Concurrency.** How many queries or requests a Data Product can serve simultaneously without degrading. High concurrency matters for user-facing products (an in-app feature hit by millions of users); less so for batch analytical products consumed by a few dashboards.

DATA FLOW TERMS

- **CDC (Change Data Capture).** An ingestion pattern that replicates changes (inserts, updates, deletes) from a source system in near-real-time, rather than periodically re-reading the whole dataset. CDC is how operational databases feed a data platform without the latency of full batch refreshes.

PROTOCOLS

- **MCP (Model Context Protocol).** An open protocol originated by Anthropic in late 2024, now broadly adopted as the standard way AI agents connect to tools and data sources. In this guide, MCP is the wire through which AI agents consume Data Products: the protocol the agent uses to query semantics, inspect schema, check quality, and read product metadata.

Position

Data is not a byproduct of the systems that generate it. It is an asset, and when it is served to a consumer (a human, a pipeline, an AI model, an application), it is served as a *product*: with an owner, a contract, a consumer in mind, and a lifecycle that is managed deliberately.

This guide sets out what that means in practice. It defines the concept, the design principles that guide it, the architectural choices that follow from them, and the anti-patterns it refuses to repeat. It is meant to be short, opinionated, and binding: a shared frame for how Data Products are built, not an encyclopedia of options.

The position exists because the alternative (data treated as whatever happens to exist in the warehouse) has measurable costs. Pipelines break undetected. Models are trained on inputs no one can reconstruct. Three teams define "active customer" three different ways. Dashboards disagree, and the disagreement is investigated instead of the business question. These costs are paid by every organization that lets them accumulate. This guide intends to stop paying them.

What follows is not a tooling manifesto. It does not prescribe specific vendors, team structures, or which products get built first. It takes a position on the *shape* of a Data Product, the *foundation* it sits on, and the *discipline* required to keep both credible over time. Tooling and process follow from that, not the other way around.

The discipline this guide prescribes is, at its core, software engineering applied to data. Contracts before implementation. Versioning with backward-compatibility. Tests that run in CI and block bad releases. Code review. Deprecation windows. Rollbacks as a runbook rather than a crisis. Observability as first-class output. None of these practices is invented here; they are what any capable software team would do if data were its product. Data work has often been treated as a craft that sits outside this discipline: a query written and run, a dashboard built and forgotten, a pipeline that succeeds often enough. A Data Product rejects that positioning.

A companion external piece, "[Data Products: The Essential Context for Enterprise AI](#)," makes the market-facing case for why Data Products matter in the AI era. This guide defines the internal standard.

PART ONE

Principles

Definition

A **Data Product** is a self-contained, managed unit of data that's treated like a product (with owners, consumers both human *and* machine, SLAs, and a lifecycle) rather than as a byproduct of some operational system.

Core properties

- **Purpose-built.** It solves a specific analytical, operational, or AI need, not "data in general."
- **Owned.** A named owner is accountable for its quality, evolution, and deprecation.
- **Discoverable and addressable.** Findable in a catalog, accessed via a stable interface (table, API, topic, file path) that agents and pipelines can resolve without human help.
- **Trustworthy.** Published, *machine-readable* guarantees on freshness, accuracy, completeness, and availability that downstream jobs can gate on.
- **Self-describing.** Schema, semantics, lineage, and usage examples travel with the data, so both people and LLMs can reason about it correctly.
- **Interoperable.** Standard formats and shared identifiers, so products compose cleanly across training, features, and inference.
- **Versioned.** Schema and semantic changes are managed like API changes, not silent breakages that poison a model undetected.

Why this shape is AI-ready

- Context travels with the product. Column meanings, status codes, business rules, and definitions are carried by the asset itself, not scattered across documentation, ticketing systems, or tribal knowledge. Consumers should not need to reconstruct context from elsewhere to use the product correctly.
- A semantic contract kills train/serve skew: the same definition of "active customer" or "net revenue" powers offline training, online features, and agent queries.
- Trust signals as data allow RAG refreshes, feature pipelines, and autonomous agents to decide whether today's data meets the bar, the only way AI runs safely without a human in the loop.
- Grounded retrieval needs curated, defined inputs. Point an LLM at a Data Product, get explainable answers; point it at raw warehouse tables, get confident fabrication.
- Lineage and classification are intrinsic, so "what trained this model, and is any of it PII?" is answerable by reading the asset, not by forensics after an incident.

AI-ready is not upgraded analytics-ready

A common assumption worth refusing up front: that **AI-ready data** is just a more mature version of analytics-ready data, and that a team with trustworthy dashboards is therefore AI-ready. It is not.

Analytics-ready data is optimized for human consumption: correct, aggregated, stable, explainable. It compresses reality into summaries humans can interrogate. AI-ready data is optimized for machine consumption: context-rich, complete, timely, semantically explicit. It expands reality into the surrounding detail a model needs to reason. Analytics removes variance as noise; AI needs variance because edge cases often carry the signal.

These are not two points on one path. They are two distinct optimisations of the same underlying substrate, and a Data Product serving AI cannot be derived by polishing the dataset that serves a dashboard. It has to be built for that consumer from the raw events forward. Teams that miss this distinction end up in what the MD101 archives call *the dashboard trap*: governed metrics, trusted reports, and AI systems that hallucinate anyway, because the data has been summarized past the point where a model can reason over it.

What a Data Product is not

- A raw table someone dumped into the warehouse
- A one-off extract or a dashboard
- A pipeline (the pipeline is plumbing; the product is the *output* plus its contract)
- A vector index bolted onto undefined source data

The mental shift

From *"here's the data, go figure it out"* to *"here's a packaged offering with a contract, a support model, and both human and machine consumers in mind."*

That definition stands on its own. It predates data mesh and applies whether you're centralized, federated, or something in between. Mesh made Data Products the atomic unit of an *operating model*; AI makes them the atomic unit of *safe consumption at machine speed*.

Design Principles

Seven principles follow from the position. Each is a direct application of software engineering discipline to data: specifications before code, API-style versioning, tests that block bad releases, deliberate design over emergent shape, measurable outcomes. None of them is novel in a software context; all of them are unusual in a data context. That's the point.

1. Start with a product spec, not a pipeline. Every Data Product begins with a written spec: who consumes it, what decisions or actions it enables, the questions it must answer, SLAs, and what's explicitly *out of scope*. No spec, no product. This single discipline prevents 80% of the sprawl and rework that plagues data teams, technically and commercially.

2. Model the domain deliberately. Data modeling is the core craft, not an afterthought. Entities, grains, relationships, and conformed dimensions have to be designed, not inherited from whatever the source system happened to emit. A good model makes the product intuitive to query, cheap to evolve, and safe to compose with others. A bad model is a tax every consumer pays forever.

3. Contract-first, implementation-second. Schema, semantics, and SLAs are declared and published before the pipeline is built. The contract is the product; the pipeline is swappable plumbing. Consumers bind to the logical interface, never to physical storage paths.

4. Fit-for-purpose scope with one clear owner. A Data Product answers a bounded set of questions well, not every possible question poorly. One team owns it end-to-end (definition, quality, evolution, deprecation). Shared ownership means no ownership; unbounded scope means no SLA can hold.

5. Quality is in-band, not after-the-fact. Validation (tests, audits, assertions) runs as part of the build and gates publication. Bad data is blocked, not alerted on. Trust signals (freshness, completeness, lineage) are emitted as first-class, machine-readable outputs so downstream consumers can gate on them too.

6. Versioned and backward-compatible by default. Schema and semantic changes follow an API-style lifecycle: versioned, deprecated, sunset. Consumers, including models in production, are never surprised. This is what makes a Data Product *durable* rather than a moving target.

7. Measured like a product. Adoption, usage, cost-to-serve, and business outcomes are tracked. Products that drive decisions get invested in; products nobody uses get retired. Without this feedback loop, you're building artifacts, not products.

Grounding a Data Product

The engine beneath a Data Product is an important decision, but not the *first* decision. Picking it up front is what causes the "federation vs materialization" debate: one that usually misses the point. The actual chain is: **ownership comes first, consumption pattern comes second, engine falls out last**. When you work it in that order, most of the argument disappears.

3.1 Ownership comes first

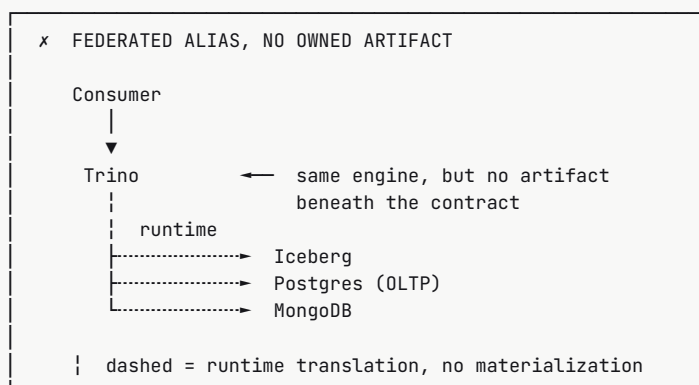
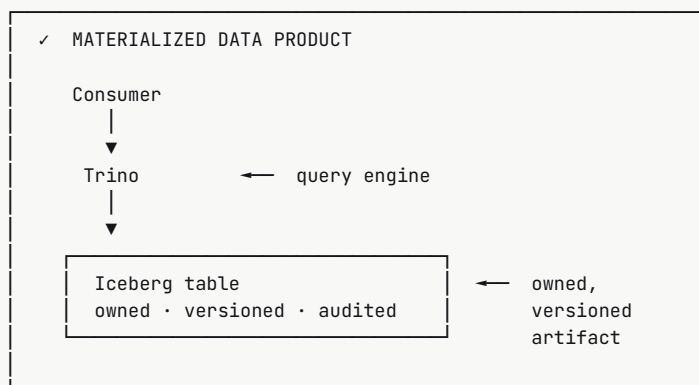
A Data Product makes promises to a consumer: a schema, an SLA, a freshness guarantee, a versioning policy. You can only make those promises if you **own a versioned artifact** that the contract binds to. Without that artifact, there is nothing durable beneath the contract, just a pass-through to whatever upstream systems happen to be doing today.

This reframes the usual debate. The axis that matters is not *federation vs materialization*. It's **ownership of a versioned artifact**. A query engine running over a table your team owns, versions, and audits is a perfectly good foundation. A query engine translating across systems you don't control at runtime is not. The test is simple: *is there a versioned, owned artifact the contract is bound to?* If yes, you have a product. If no, you have a query alias dressed up in product language.

A concrete illustration. Consider two architectures that both involve Trino:

- **Iceberg → Trino.** The Iceberg table is owned, versioned, and materialized by one team. Trino is just the query engine; the artifact beneath it is durable and under deliberate control. Schema changes go through a release cycle, lineage is intrinsic, time-travel is supported. The contract sits on something real. This is a materialized Data Product.
- **Iceberg + Postgres + Mongo → Trino.** Trino federates across three sources at runtime. No single team owns the composite. Postgres schema drift, Mongo connection saturation, and Iceberg partition changes all leak through to every consumer. There is nothing durable for a contract to bind to. It's a thin alias over moving parts.

Same query engine in both cases. Totally different products. The engine isn't the issue; ownership of a versioned artifact is.



Same query engine. Different products.
The engine isn't the issue; ownership of the artifact is.

"Materialization" is a loaded word. It sounds like "make a copy" and triggers arguments about storage cost. The frame is wrong. The thing you need is a stable, versioned, owned artifact, and that can take several shapes:

- A physical table in a warehouse or lakehouse (Parquet, Iceberg, Delta)
- An incrementally maintained materialized view with a defined refresh policy
- A versioned snapshot or time-travel table
- A zero-copy clone over immutable files

What they share: someone owns it, it has a version, it has known freshness, and its shape doesn't change beneath the consumer without a release cycle. That's what makes a contract credible.

What you lose without an owned, materialized artifact is not subtle.

- **Reproducibility.** A model trained on a virtual view cannot be retrained identically tomorrow.
- **Snapshot consistency.** Two consumers at different moments see different results.
- **Schema drift absorption.** Upstream schema changes stop being absorbed at ingestion and start leaking to every consumer at query time.
- **Performance bounds.** Can't be promised, because you don't own the layout.

- **Column-level lineage.** Has to be reconstructed from logs instead of being there by construction.
- **Access policy enforcement.** Scatters across every upstream source, and gaps appear.

The operational test: *"what did this dataset look like at 3pm yesterday, and who accessed it?"* If you can't answer deterministically, you don't have a product.

3.2 Consumption pattern comes second

Once ownership is settled, the next question is *how the product will be consumed*, because that, not the technology menu, determines what shape the artifact needs to take. A handful of patterns cover most of the ground:

- **Analytical / BI.** Wide scans, aggregations, a few seconds of latency, moderate concurrency. Humans behind dashboards and analysts behind notebooks.
- **Operational / API.** Point lookups, millisecond latency, high concurrency, sometimes transactional consistency. Applications calling for a single record or a small set.
- **Real-time analytics / user-facing.** Sub-second aggregation over fresh data, very high concurrency. Product features, live dashboards, in-app metrics.
- **AI training.** Large batch reads, strict reproducibility, full lineage. Runs periodically, tolerates minutes of latency, cannot tolerate undefined inputs.
- **AI inference / feature serving.** Millisecond reads of pre-computed features, very high concurrency, consistent with training definitions.

The consumption pattern determines freshness, latency, concurrency, and consistency requirements, and those in turn determine what engine can credibly serve the contract.

3.3 Engine choice falls out of the first two

With ownership established and the consumption pattern named, the engine decision becomes mechanical rather than philosophical. The families:

OLTP engines (Postgres, MySQL, Spanner, CockroachDB). Row-oriented, tuned for point lookups, high write concurrency, transactional consistency. The right home for operational Data Products. The wrong home for analytical scans: putting an OLTP under OLAP load melts production traffic.

OLAP engines: too coarse as one category, so split:

- *Cloud DW / lakehouse* (Snowflake, BigQuery, Redshift, Databricks, Fabric). Batch and near-real-time analytics, wide scans, large volumes. The workhorse for analytical products, AI training datasets, BI.
- *Real-time / serving OLAP* (ClickHouse, Druid, Pinot, StarRocks). Sub-second aggregation over fresh data at high concurrency. User-facing analytics, operational dashboards, ML feature serving.

- *Embedded OLAP* (DuckDB, Polars). In-process, single-node. CI tests, local development, small products.

Federation engines (Trino/Starburst, Dremio, Denodo, Spark SQL federation). Query-time translation, no owned storage. The right home for exploratory cross-source queries, bootstrap before a canonical model exists, thin SQL layers over already-governed stable sources, and cost-optimized access to cold data. The wrong home as the foundation of a consumer-facing product, because they own no artifact, so nothing durable sits beneath the contract. See the Iceberg-vs-multi-source contrast in §3.1 for why the same engine can be either a good or a bad foundation depending on what sits beneath it.

The decision reduces to matching the consumption pattern to the family: milliseconds and point lookups → OLTP or serving OLAP; seconds and wide scans → cloud DW; sub-second aggregation at high concurrency → serving OLAP; reproducible batch reads for training → lakehouse. Volume, write pattern, and cost shape the specific pick inside each family.

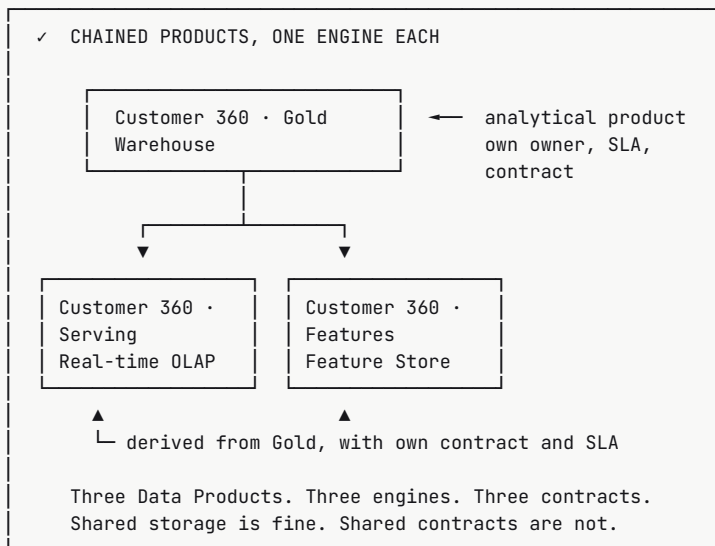
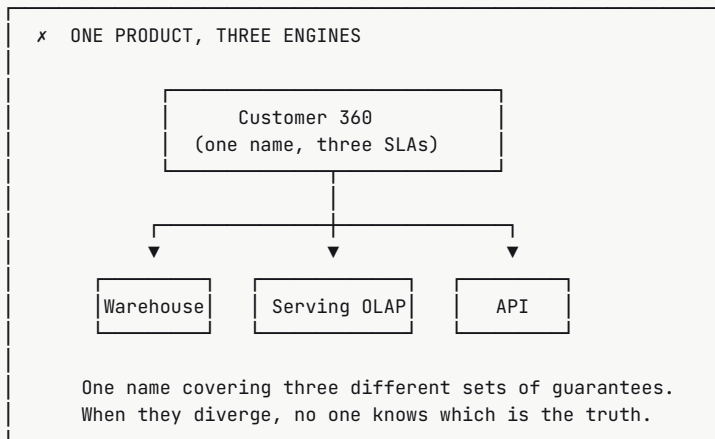
3.4 One Data Product, one engine

A tempting mistake is to serve a single Data Product from multiple engines, the same "customer 360" exposed as a warehouse table *and* a low-latency API *and* a real-time feature. It looks efficient. It's not.

A Data Product has **one contract**: one SLA, one freshness guarantee, one consistency model, one performance envelope. Those guarantees are engine-dependent. Two engines serving the same product mean two different sets of guarantees living under one name. That is not one product; it is two products pretending to be one. When they diverge (and they will), no one knows which was the source of truth at which time, and the contract stops meaning anything.

The right pattern when multiple consumption shapes exist is to build **multiple Data Products, chained**. A gold analytical product lives in the warehouse. A serving product for the API is derived from it, materialized into a serving OLAP engine, with its own owner and its own contract. A feature product for inference is derived similarly, with its own SLA and its own freshness guarantee. Each has one engine, one contract, one owner. The lineage between them is explicit, and when the serving product is stale, the contract says so instead of pretending otherwise.

This also resolves the common objection "*but Iceberg can be queried by many engines!*" Yes, that's the storage layer. A Data Product chooses **one** engine to expose to consumers through its contract. Other teams may query the same underlying storage to build *their own* products with their own contracts. Those are different products, even if they share bytes on disk.



The principle: **one Data Product, one engine, one contract**. Shared storage is fine. Shared contracts are not.

Anti-patterns to Refuse

A manifesto is as much about what it rules out as what it endorses. The following shapes are committed against, not because each is always wrong in isolation, but because calling them Data Products dilutes the term until it means nothing.

The data swamp. A warehouse full of tables no one owns, documents, or can confidently use. Rejected because an uncurated warehouse is not a platform. It's a cost center that erodes trust every quarter it goes untended.

The dashboard-as-product. A BI dashboard treated as the deliverable, with no underlying artifact other consumers can use. Rejected because a dashboard is a *view* of a product, not the product itself. Without the artifact beneath, the logic lives in the dashboard and cannot be reused, audited, or evolved.

The federated alias. A "Data Product" that is actually a runtime query over sources the producing team does not own: a view with a nicer name. Rejected because nothing durable sits beneath the contract, and the promises it makes are promises someone else has to keep without knowing they made them.

The shared contract across engines. One Data Product served from a warehouse, a serving OLAP, and an API (under a single name, as if the guarantees were the same). Rejected because one name and multiple SLAs is two products pretending to be one, and the pretense breaks the first time they diverge.

The permanent beta. A product that has been "in alpha" for two years, used in production, with no owner willing to commit to an SLA. Rejected because undefined maturity is a governance gap: consumers assume production, producers assume experiment, and the gap is filled with incident reports.

The orphaned product. A product whose original team disbanded or moved on, still running, still consumed, with no one empowered to change or deprecate it. Rejected because ownership is a property of *now*, not of history. An unowned product must be re-owned or retired, never left running on momentum.

Governance by PDF. Policies, definitions, and classifications that live in documents no pipeline can read. Rejected because governance that cannot be queried cannot be enforced, and in a world of AI consumers, unenforceable governance is no governance at all.

The pipeline mistaken for the product. The transformation job is shipped, scheduled, and monitored. But there is no contract, no consumer in mind, and no ownership of the output. Rejected because the pipeline is plumbing. The product is the *output plus its contract*, and without the second half, infrastructure has been built for nobody.

What This Position Costs

The position in this guide is not free. Adopting it has costs (in time, in storage, in discipline) that treating data as exhaust does not. Those costs are named explicitly here, because a position whose costs are hidden is a position that will be abandoned the first time it is inconvenient.

Slower time to first query. A Data Product begins with a spec, a contract, and a deliberate model, not a `CREATE TABLE AS SELECT`. For one-off questions, this is overhead. The cost is accepted because *most* data work is not one-off, and the cost of skipping the spec compounds across every future consumer.

Storage duplication: mostly a myth, but worth naming. A materialized artifact takes more storage than a virtual view. In a modern data stack, storage is the cheapest component (orders of magnitude cheaper than compute or engineering time) and the reproducibility, consistency, and performance bounds a materialized artifact buys are not comparable in value. The trade-off is named to be honest about it, not because it is close.

Fewer products, more deliberately built. A team applying these standards ships fewer Data Products per quarter than a team shipping raw tables. This is a feature, not a bug. A smaller number of trusted products consumed by many is worth more than a large number of untrusted tables consumed cautiously.

Ongoing maintenance, not one-time build. Products have lifecycles. They require owners, SLA monitoring, version management, deprecation communication. A Data Product is never "done" the way a one-off extract is. This cost is accepted because the alternative (a warehouse of artifacts in unknown states) is the cost this guide exists to stop paying.

Political cost of saying no. Applying these standards sometimes means refusing to call something a Data Product when a stakeholder wants the label. It means pushing back on "just expose this table" requests. The friction is real and it falls on producing teams. The cost is accepted because the label means something only if it is withheld when it doesn't fit.

Up-front modeling cost. Deliberate data modeling takes longer than reflecting whatever shape the source system produced. The payoff is downstream (in query intuitiveness, evolution cost, and composability), but the cost is borne up front by the producing team. The cost is accepted because a bad model is a tax every consumer pays forever, and up-front design is cheaper than retroactive refactoring.

Is This a Data Product? A Decision Rubric

The preceding sections define what a Data Product is and what it rests on. This section turns that definition into a gate. A proposal or existing artifact is a Data Product if, and only if, it passes all eight of the following. Anything missing means it is something else: a dataset, a pipeline output, a view, a prototype. Those are all legitimate things. They are not Data Products, and should not be labeled or governed as such.

IDENTITY AND INTENT

1. **Named owner.** A single team or individual is accountable for quality, evolution, and deprecation. "The data team" is not an owner. A name is.
2. **Written spec.** The product has a document describing its consumers, the decisions or actions it enables, the questions it answers, and what is explicitly out of scope. If the spec does not exist in writing, the product does not exist.
3. **Identified consumers.** At least one named consumer (human, pipeline, model, or application) exists and has agreed the product meets their need. A hypothetical future consumer does not count.

FOUNDATION

1. **Owned artifact.** A versioned, materialized artifact sits beneath the contract. Not a virtual view over sources the producing team does not control. Not a federated alias. An artifact.
2. **One engine.** The product is served through a single engine with a single SLA and a single consistency model. Multiple consumption shapes mean multiple chained products, each with its own contract, not one product with one name and two sets of guarantees.

CONTRACT

1. **Published contract.** Schema, semantics, SLA, freshness, versioning policy, and support model are documented and discoverable, not in a PDF no pipeline can read.
2. **Quality gates in place.** Tests, audits, and assertions run as part of the build and block publication of bad data. After-the-fact alerting does not qualify.
3. **Versioning policy.** A stated policy for how breaking changes are handled: deprecation windows, consumer migration, version retirement. "We'll coordinate" is not a policy.

A product that passes all eight is a Data Product. A product that passes five or six is a candidate: something to bring to this bar, not something to ship under the label. A product that passes fewer is a dataset, a view, or a pipeline output, and that is what it should be called.

How this is used. When a team proposes something as a Data Product, this rubric is the review checklist. When a team inherits an existing asset and is unsure how to treat it, this rubric is the audit. When a stakeholder disputes the label, this rubric is the arbiter. The point is not gatekeeping. It is clarity. Labels carry expectations; expectations carry obligations. A Data Product's label should carry both honestly.

Data Product Tiers

Not every Data Product warrants the same rigor. A product feeding regulatory reporting and three production ML models must clear a higher bar than a product serving one team's internal dashboard. A single standard applied to both either over-engineers the second or under-protects the first. Neither serves the organization. Three tiers are defined below, with differentiated obligations. Every Data Product belongs to exactly one.

Tier 1: Critical

TIER 1

Products whose failure has material business, regulatory, or customer consequences. Examples: financial reporting inputs, customer-facing data served in production applications, inputs to models making consequential decisions (credit, fraud, medical, pricing), and any data subject to regulatory audit.

- Passes the full §6 rubric without exception.
- SLA is contractual, measured continuously, and reported.
- Quality gates include completeness, accuracy, and reconciliation checks, not just schema validation.
- Breaking changes require a minimum deprecation window and explicit consumer sign-off.
- Reviewed at inception by Platform / IT (architectural consultation on the substrate) and by Governance (policy and classification), and re-reviewed by Governance at least annually.
- Documented lineage end-to-end, including upstream systems.

Tier 2:

TIER 2

Core

Products consumed by multiple teams or domains, where correctness and stability are important but not regulatory or customer-visible. Examples: shared entity tables (customer, product, order), metrics used across domains, feature datasets used by multiple ML projects.

- Passes the full §6 rubric.
- SLA is published and monitored, but measured less intensively than Tier 1.
- Quality gates include schema and completeness; accuracy checks where feasible.
- Breaking changes require a deprecation window but not formal consumer sign-off.
- Architectural consultation with Platform / IT at inception, where the substrate or capacity needs warrant it.
- Documented lineage to named upstream sources.

Tier 3:

TIER 3

Local

Products consumed by a single team for a bounded purpose. Examples: a team's internal dashboard data, exploratory products still maturing, domain-specific aggregations used by one analytics group.

- Passes the §6 rubric, with the caveat that the contract and consumer set can be lighter.
- No published SLA required; best-effort freshness documented.
- Quality gates include schema validation at minimum.
- Breaking changes communicated to known consumers; no formal deprecation window required.
- No external review required, but registered in the catalog.

Moving between tiers

Tiers are not static. A Tier 3 product that accumulates cross-team consumers graduates to Tier 2. A Tier 2 product adopted by a Tier 1 system either pulls its obligations up or gets forked into a Tier 1 variant with tighter guarantees. The reverse, downgrading a Tier 1 product, is rare and requires explicit governance review, because consumers have built on the stronger guarantees.

What the tiers are not

Tiers are not a quality ranking. A Tier 3 product is not lower-quality than a Tier 1 product within its scope; it simply has a smaller scope and correspondingly lighter obligations. A Tier 3 product that fails its local consumers is just as broken as a Tier 1 product that fails regulatory reporting; the *blast radius* is what differs, not the standard within each tier.

Roles and Responsibilities

A Data Product is only as owned as the roles around it. Vague accountability is the single most common failure mode: "the data team owns it" is not ownership, it's diffusion. Three roles are defined here. Every Data Product has one instance of each; the individuals filling each role may differ from product to product, but the roles themselves are fixed.

The Data Product Team

One team builds, runs, supports, and evolves the product end-to-end. The shape mirrors a software service team: the team owns the code (transformations, semantic model, validation), ships releases (versioned materializations), runs the service (observability, on-call, incident response), supports its consumers (contract questions, deprecation notices), and plans its evolution. "Owns end-to-end" is not a slogan; it's the same end-to-end ownership a microservice team has over its service. The team is the single point of contact for anything related to this product, consumers, stakeholders, and downstream teams engage the Data Product Team, not a collection of functions spread across an organization.

The team is *not* responsible for running the underlying infrastructure: the warehouse, the lakehouse, the serving engines, the orchestrator. It consumes those the way any engineering team consumes infrastructure: requesting capacity, reporting issues, escalating when the substrate is the problem. It is responsible for everything *above* that line, from the contract to the consumer.

The Data Product Lead

A named individual *inside* the Data Product Team who owns the contract and is the accountable face of the product externally. Sets the SLA, approves or rejects consumer requests for changes, declares deprecation, and makes the final call when the contract is in dispute. When a consumer has a question about what the product does or will do, the Lead is the person they talk to.

The Lead is not a separate role filled by a separate person. It is a hat worn by one team member. Everyone on the team builds, runs, and supports the product; the Lead additionally carries the external accountability. This keeps the team cohesive while preserving the discipline that *one name* sits on the contract.

The Lead is *not* a hierarchical position. They do not manage the team in a line-management sense. They are the accountable interface with the outside world.

Governance

The function (often a named team, sometimes a virtual committee) accountable for the policies that cross all Data Products: classification (PII, sensitive, restricted), access controls, retention, regulatory compliance, and the rubric and tier definitions themselves. Reviews Tier 1 products at inception and annually. Adjudicates disputes that cross product boundaries or require policy interpretation.

Data Product Teams operate *within* the policies Governance sets; they do not inherit or re-invent those policies per product. Governance is *not* a co-owner of any product and *not* a blocker to be routed around. It is the function that makes cross-product guarantees possible. Without it, every product's access policy and classification is re-invented, and an organization has no defensible position on compliance.

Where Platform / IT fits

The Data Product Team engages Platform / IT the way any engineering team engages an infrastructure provider. Capacity is requested; incidents affecting the substrate are escalated; upgrades and migrations are coordinated. Platform / IT is a *supplier* to the product, not a *co-owner* of it. The team owns the product; Platform owns the substrate beneath it. The line between them is the same line that separates any service team from its infrastructure provider, and it should be kept crisp for the same reason: shared ownership across that line is what produces the "no one's accountable when it breaks" failure mode this section exists to prevent.

How disputes are resolved

Most disputes about a Data Product fall into one of three categories, and each has a clear resolution path:

- **Contract disputes** (what the product should do, how it should behave, what the SLA should be) are resolved by the Data Product Lead. The Lead is the single point of authority on the contract.
- **Execution disputes** (whether the product is meeting its contract) are resolved within the Data Product Team. If the root cause is infrastructural, the team escalates to Platform / IT, but the team remains accountable to the consumer throughout.
- **Policy disputes** (classification, access, compliance, tier assignment) are resolved by Governance. This is final.

PART TWO

Implementation

Part 1 set out the principles. This part describes how DataOS makes them real. Every capability described here traces back to a principle in Part 1. Readers building a Data Product in DataOS will most often reach for §4 (Production) and §5 (Consumption); readers evaluating an adoption shape will most often reach for §8 (Full-stack and Overlay Patterns); readers tracking the cross-product architecture roadmap will most often reach for §7 (the Semantic Layer).

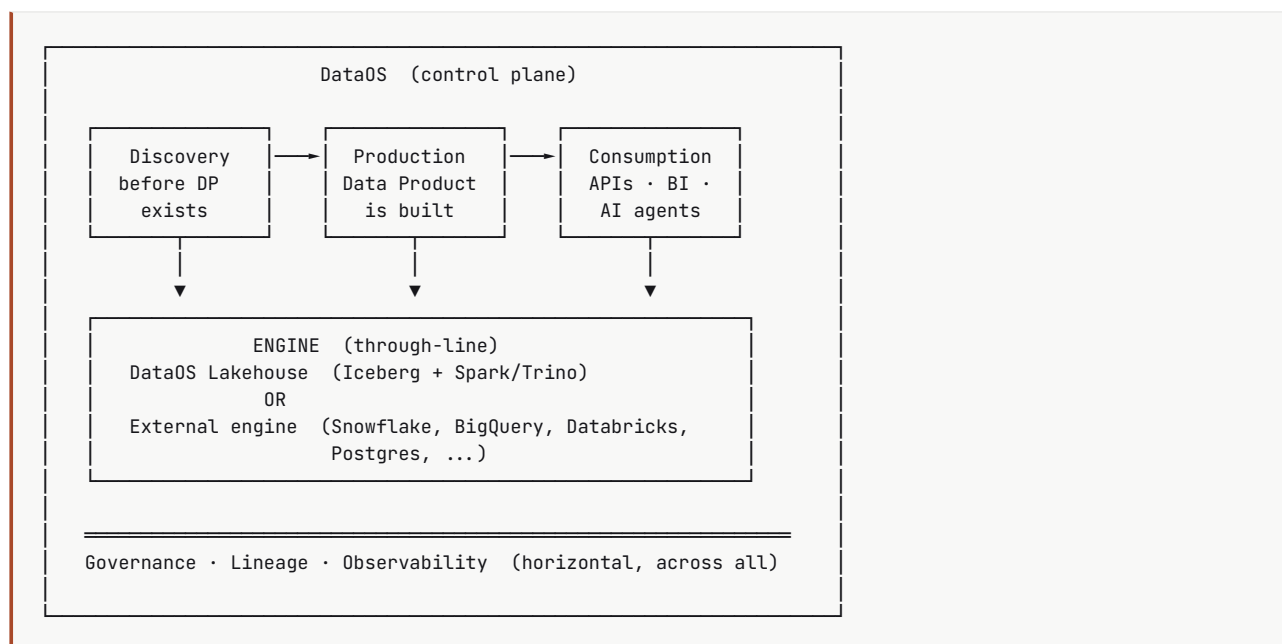
PART 2 · §1

The Three-Stage Flow

A Data Product in DataOS is supported across three stages: **Discovery**, **Production**, and **Consumption**. Discovery happens before any specific Data Product exists, when a team is answering "*what data do we have, and can we build what we want from it?*" Production is where a decided-upon Data Product is built in the chosen engine. Consumption is where the built Data Product is used by humans, applications, and agents.

Strictly, only Production and Consumption are stages of a Data Product's lifecycle. Discovery is what happens *before* a Data Product exists at all. The three stages are grouped here because the capabilities supporting them all live in DataOS, and a team moves through them in sequence when building something new.

The engine is the through-line across all three. In Discovery, a team inspects what lives in the engine (and brings in what's missing). In Production, Data Products are built in the engine. In Consumption, consumers read from the engine. DataOS is the control plane; the engine is where the bytes actually live.



This path is the whole of DataOS. Every feature described in the sections that follow exists to make one of these three stages real: to answer *what do we have and can we build from it* (Discovery), to build a Data Product reliably (Production), or to put that product in front of its consumers (Consumption). Features that fall outside this path are not part of DataOS. The coherence of the platform comes from that constraint: discipline about what belongs, not accumulation of what could.

Horizontal capabilities (governance, lineage, observability) apply uniformly across all three stages, and are covered in §6.

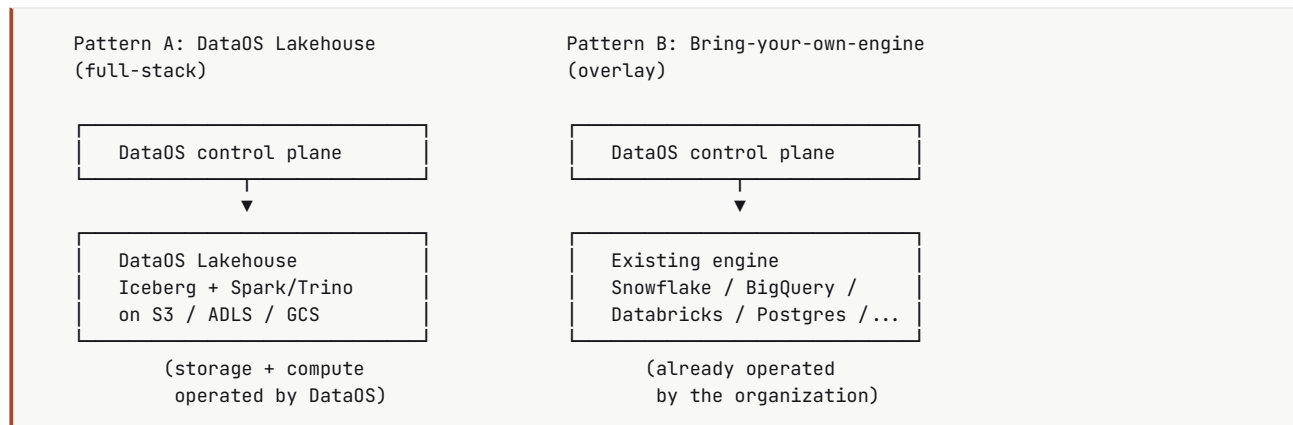
PART 2 · §2

The Engine: Through-line Across All Stages

A Data Product materializes somewhere. That somewhere is the engine. DataOS does not replace engines; it operates on top of them. Two patterns exist, and both are first-class.

Pattern A: DataOS Lakehouse. DataOS provides a first-party Lakehouse built on **Iceberg** as the storage format, with **Spark** and **Trino** as the execution engines on top. The underlying object store can be **S3**, **ADLS**, or **GCS** depending on the cloud. Teams that choose this pattern get a Lakehouse DataOS governs natively: storage, compute, and governance in one coherent stack.

Pattern B: Bring-your-own-engine. Many organizations already have data landed in a governed warehouse: **Snowflake**, **BigQuery**, **Databricks**, **Postgres**, or similar, often populated by existing pipelines like **Fivetran**, **Airbyte**, or internal **ETL**. DataOS builds Data Products directly in that existing engine, without requiring a migration to the DataOS Lakehouse.



The rest of Part 2 sits above this choice. §3 through §6 apply to both patterns, because engine is deployment, not discipline. §7 sits above the engine layer entirely, composing Data Products regardless of where they were built. §8 covers how to choose between the two patterns.

PART 2 · §3

Discovery, Before the Data Product Exists

Discovery is the work a team does before any specific Data Product exists. It answers one question: *"what data do we have, and can we build what we want from it?"* No artifacts are produced at this stage. No contracts are signed. What Discovery produces is a decision: either *yes, we have what we need, let's build* (and proceed to §4), or *we know what's missing and how to get it, or we're looking in the wrong place and need to rethink*.

This is why Discovery is not strictly part of a Data Product's lifecycle. A Data Product doesn't exist yet when Discovery is happening; the team is deciding whether one *should*. But it's critical work, and DataOS supports it with three capability clusters.

3.1 Understanding what's there: metadata, lineage, profile

The starting point for Discovery is *reading before querying*. Before running a single SQL query, a team can use **Metis** to browse what DataOS already knows about the data landscape:

- **Metadata** across connected sources and DataOS-internal resources: schemas, tables, columns, types, descriptions, tags, business terms.
- **Lineage** traced back to source systems, across any pipelines DataOS has observed. A team can see where a candidate dataset originated and what has transformed it.
- **Profile information** produced by earlier scanning: cardinalities, null rates, distributions, sample values. A team learns the shape of the data before touching it.

Metis turns a directionless search ("is there customer revenue data somewhere?") into a directed inspection ("here are three datasets that look relevant, with their profiles and lineage").

3.2 Exploring the data: SQL and EDA

Once candidate datasets are identified, the next step is interrogating them. **Workbench** is where this happens: an interactive SQL and exploration environment that connects to whatever the team has access to (raw sources, already-landed data, DataOS-internal datasets). The team writes queries, inspects samples, validates hypotheses, checks edge cases.

This is classic exploratory data analysis, but grounded in the DataOS environment: access is governed, lineage is captured, and queries run against the real substrate rather than a disconnected copy.

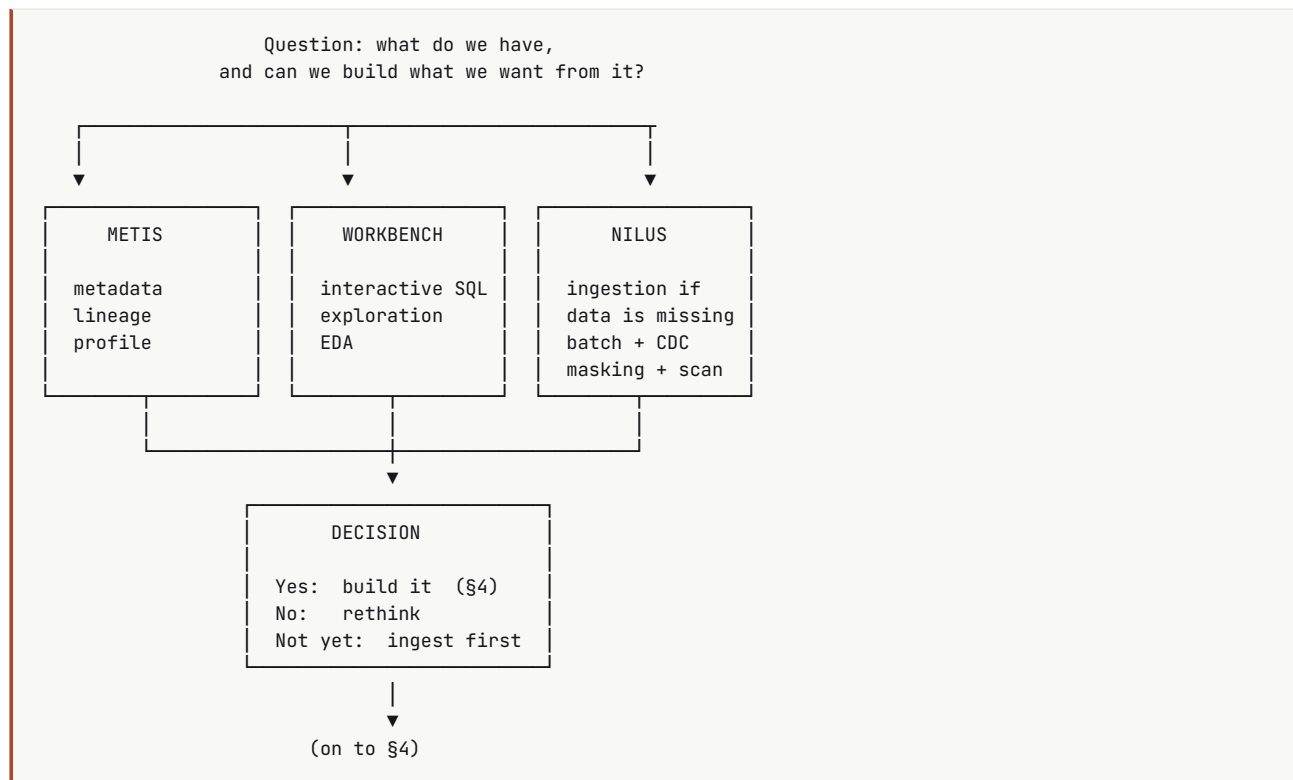
3.3 Bringing data in (if it isn't there): ingestion

If Discovery concludes that the team needs data not currently in the engine, the third cluster applies: Nilus handles ingestion.

- **Batch and CDC ingestion.** Batch for periodic pulls; Change Data Capture for near-real-time replication from transactional sources. One framework covers both.
- **Wide source coverage.** Operational databases (Postgres, MySQL, MongoDB, MS SQL Server, DB2), warehouses (Snowflake, BigQuery, Databricks, Redshift), streaming (Kafka, NATS), SaaS APIs (Salesforce, HubSpot, Stripe, Google Analytics), and custom connectors authorable in Python.
- **Schema absorption at the boundary.** Upstream schema changes are handled at ingestion, not leaked to downstream consumers.
- **Data masking for sensitive fields.** Sensitive values are transformed at ingestion, before they ever reach the governed destination.
- **Metadata scanning and profiling.** Newly ingested data is scanned, profiled, and catalogued, which feeds straight back into Metis and strengthens the next round of Discovery.

Ingestion is the *remedial* part of Discovery: something the team does when they've determined that what they need isn't already in the engine. Often ingestion is *not* needed, because the data is already there (populated by an existing pipeline, or already in the DataOS Lakehouse from a previous project).

The shape of Discovery



The three clusters are not a linear sequence. A team might start in Metis, dip into Workbench to verify, realize data is missing, trigger Nilus, wait for data to land, re-scan through Metis, explore again in Workbench, and only then decide. Discovery is iterative, and DataOS is shaped to support that iteration rather than enforce a workflow.

What Discovery is not

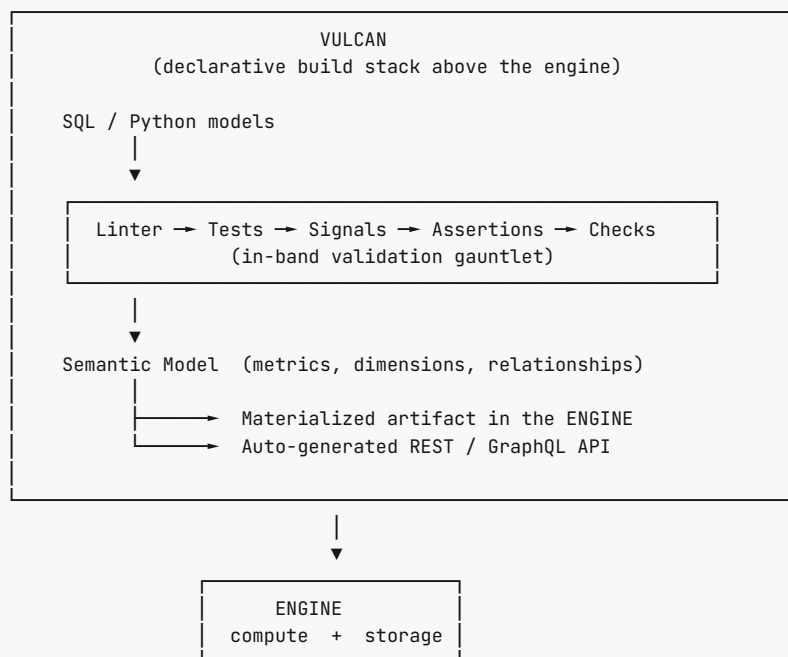
Discovery is not Production. Nothing built in Discovery is a Data Product. Queries written in Workbench are not published artifacts. Data landed by Nilus becomes a raw input to Production, not a Data Product in its own right. The moment the team decides "*yes, let's build,*" they leave Discovery and begin §4 with that decision as input.

Production, Building the Data Product

Production is where Data Products are built. The engine hosts the data and runs the compute; the build stack above the engine is **Vulcan**. Vulcan turns raw landed data into a versioned, validated, contracted Data Product with a thin API layer for consumers.

The capabilities at this stage:

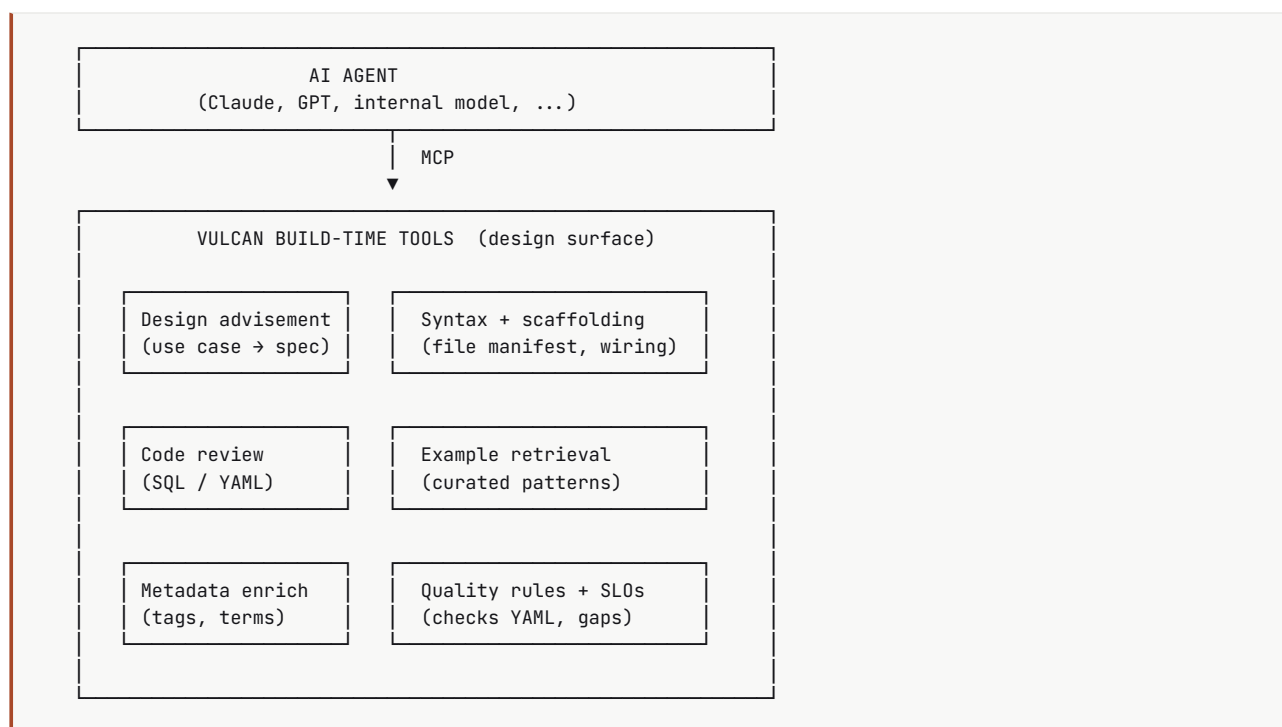
- **Declarative transformations in SQL or Python.** SQL for the bulk of transformations; Python for logic that is painful in SQL (API calls, ML scoring, complex business rules). Both coexist in one project.
- **In-band validation.** Transformations pass through a validation gauntlet before publication: a linter catches syntax errors, tests confirm expected outputs, signals verify dependencies are met, assertions and quality checks block bad data from being published. The Part 1 principle *quality is in-band, not after-the-fact* is enforced here by construction.
- **Semantic model.** The Data Product's own bounded context: metrics, dimensions, entities, and relationships declared once and consumed everywhere the product is accessed. The semantic definition is what consumers bind to, not raw tables. Train/serve skew is eliminated because offline analysis, online features, and AI agents all resolve through the same semantic definitions. Each Data Product's model is local and self-contained; how models compose across products is addressed separately in §7.
- **Materialization into the engine.** The Data Product is materialized as an artifact in the chosen engine: an Iceberg table in the Lakehouse, or a table in Snowflake, BigQuery, Databricks, whichever applies. This is the versioned, owned artifact Part 1 §3 requires.
- **Thin API layer for accelerated access.** Vulcan auto-generates REST and GraphQL interfaces from the semantic model. Consumers read the API rather than running raw SQL, and the API accelerates access patterns (caching, pushdown, semantic resolution) that would be slow or error-prone as ad-hoc queries.
- **CI/CD with plan-style previews.** Changes are planned, diffed against the current state, reviewed, approved, and rolled back if needed. Breaking changes follow the API-style versioning discipline Part 1 §2 requires.
- **Policy and governance at product level.** Classification, access control, and retention are properties of the product, enforced in one place. See §6.
- **Observability as first-class output.** Freshness, quality signals, run history, and lineage are emitted as machine-readable signals, not dashboard-only artifacts.



4.1 AI-assisted Data Product construction

A distinctive capability of Vulcan at this stage: an AI agent can participate in building a Data Product, not just consuming one. This is possible because a Data Product is structured as software, code, configuration, tests, semantics, all in files that can be read, written, and reviewed. An agent invoking Vulcan's build-time tools is doing the same work a human engineer does, through the same interfaces, under the same review discipline. Through an MCP interface, Vulcan exposes a set of build-time tools the agent can invoke:

- **Concept explanation and syntax templates** so an agent can scaffold correct Vulcan components without guessing.
- **Code review for SQL and YAML** so the agent can validate a human's draft before it enters the validation gauntlet.
- **Retrieval of real working examples** from curated Vulcan projects so the agent grounds its suggestions in patterns that work.
- **Design advisement** so an agent can turn a use case ("I need a customer churn metric for quarterly reporting") into a structured Data Product design specification.
- **Scaffold generation** producing a file manifest covering seed, staging, final, semantics, checks, and tests, wired together correctly.
- **Metadata enrichment** adding column descriptions, tags, business terms, and check stubs to an existing project.
- **Quality rule suggestion** producing a checks YAML, SLO definitions, audit suggestions, and a coverage gap analysis.



This is distinct from the runtime agent tools in §5. These tools *help build* a Data Product; those tools *help use* one. Both matter, and most platforms only offer the second.

4.2 Production is a lifecycle, not a one-shot build

A Data Product's first release is not its final state. Like any piece of software, it enters an iteration loop the moment it ships: consumers use it, contracts tighten, edge cases surface, metrics reveal where it under- or over-serves, business definitions shift, and new consumers arrive with requirements the original spec didn't anticipate. Production, as the term is used in this guide, covers the whole of that lifecycle: initial build, observation, iteration, versioned evolution, and eventual deprecation. The same Vulcan capabilities described above (declarative transformations, in-band validation, semantic model, CI/CD with plan-style previews, observability as first-class output) are the discipline through which a Data Product evolves, not just how it is first constructed.

The engineering loop is the familiar one from software: **design, develop, test, release, observe, iterate**. Observability emits the signals (freshness breaches, usage patterns, cost drift, consumer-reported issues) that trigger the next iteration. The semantic model absorbs definition changes without forcing consumers onto new endpoints. CI/CD carries the change through the validation gauntlet. Versioning with backward-compatibility protects consumers from surprise. The Data Product Lead (Part 1 §8) is accountable for deciding when observation has surfaced enough to warrant a new release, what goes in it, and when breaking changes require a deprecation window.

What makes this work at scale is that every step of the loop (designing the change, validating it, releasing it, observing its effect) happens inside the same Vulcan project, against the same engine, under the same contract. There is no separate "maintenance mode" distinct

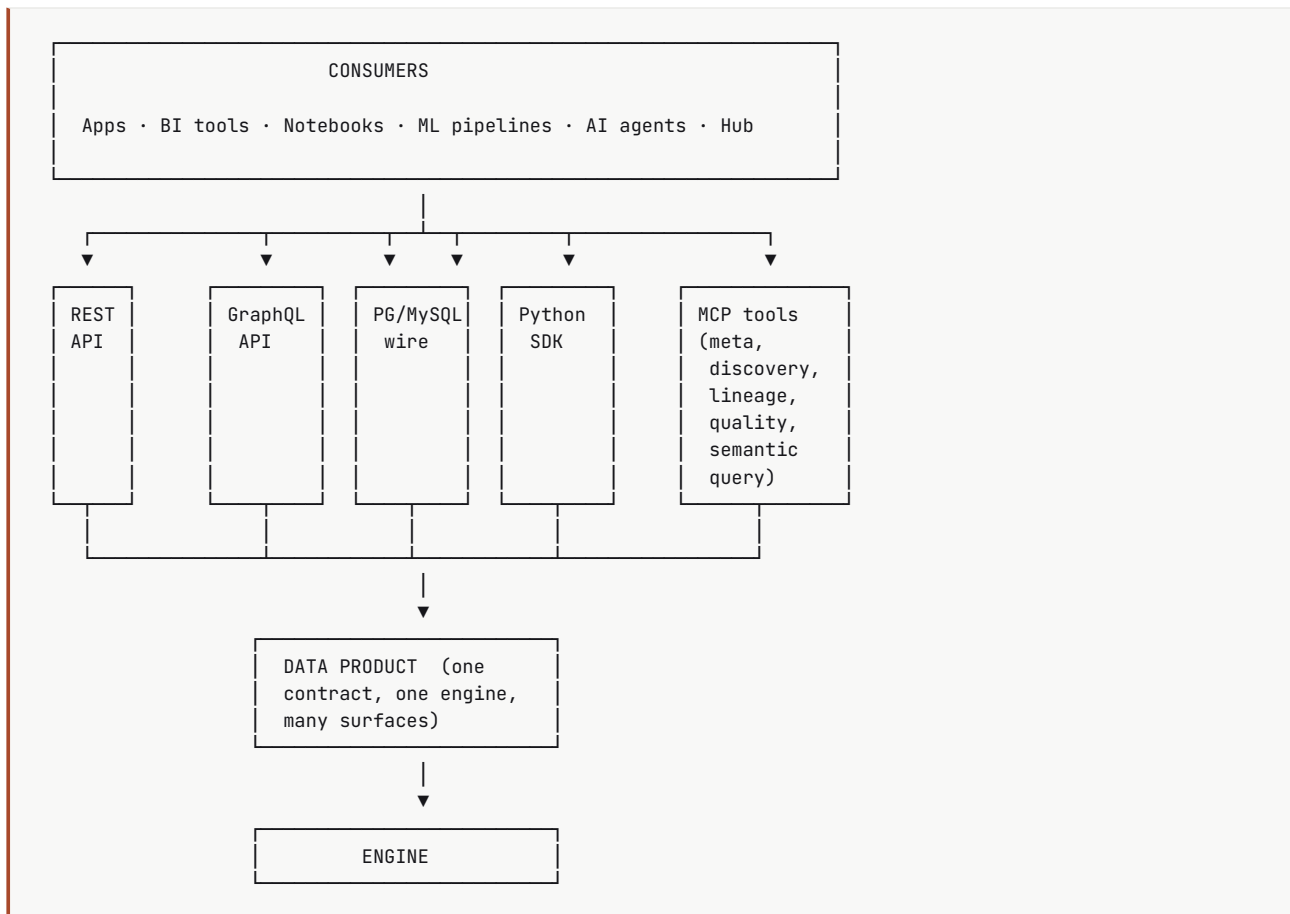
from "build mode." A Data Product under active evolution is a Data Product in Production, and the word means both *in service* and *in continued development*, the way it does for any software system that matters.

Consumption, Accessing the Data Product

Once a Data Product is built, it is accessed. DataOS exposes a Data Product through several parallel surfaces, each matched to a type of consumer. The contract behind each surface is the same (one product, one engine, one set of guarantees, per the [Part 1 §3.4](#) principle); the protocol adapts to the consumer.

The surfaces:

- **Thin REST and GraphQL APIs** auto-generated from the semantic model. Applications, services, and modern frontends call these directly.
- **Database wire protocols (Postgres, MySQL)** so BI tools and SQL clients connect as if the Data Product were a regular database. Tableau, Power BI, Superset, and Excel all reach Data Products this way.
- **SDKs and notebook access** for analysts and ML practitioners. Python SDKs expose Data Products to Jupyter, training pipelines, and custom applications.
- **MCP runtime tools for AI agents.** A set of structured tools through which an agent can query semantics, inspect schema, check quality and freshness, view run history, and read product metadata. This is the agent-equivalent of what BI does for humans, except machine-readable and semantically grounded.
- **Data Product Hub for discovery and activation.** A catalog surface where humans find Data Products, understand their contracts, and activate them against the BI or ML tool of their choice.



The MCP tools are abstracted above as "metadata, discovery, lineage, quality, semantic query" because that captures what they let an agent do. The individual tool names evolve; the capability they collectively provide is what matters for this guide: an AI agent can *safely* use a Data Product with the same contractual guarantees a human gets, because it resolves the product through its semantic model rather than scraping raw storage.

PART 2 · §6

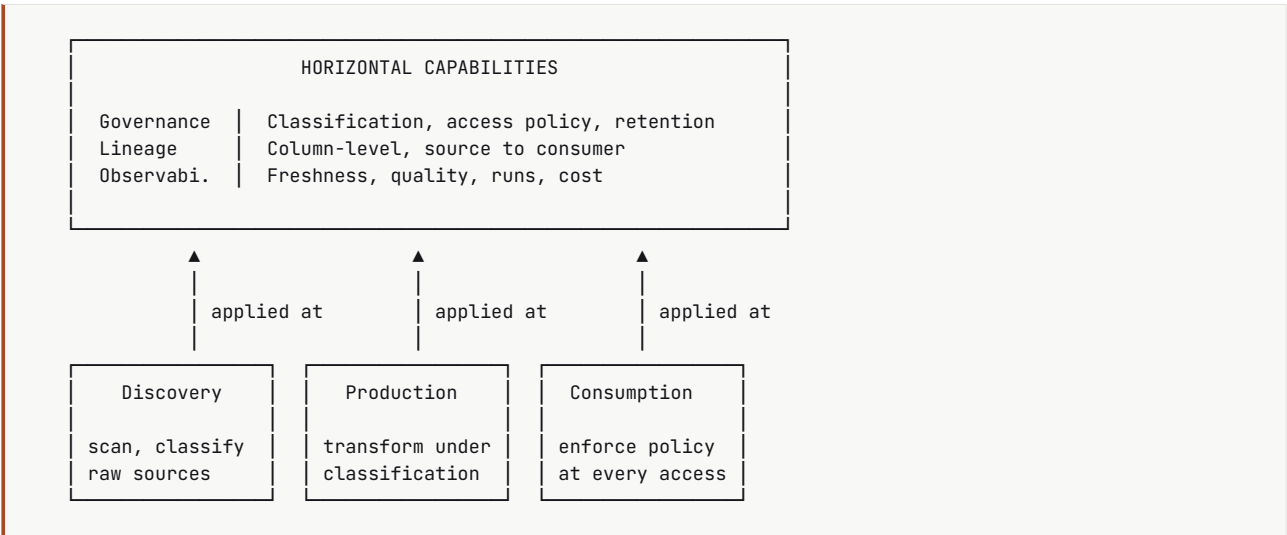
Governance, Lineage, Observability Across All Stages

The three stages describe how a Data Product comes into existence and how it is consumed. Three horizontal capabilities wrap around all three, and apply uniformly regardless of which stage is active.

Governance. Classification (PII, sensitive, restricted) is applied to data as it is discovered and ingested, propagates through Production, and enforces at Consumption. Access control, row-level and column-level policies, and retention are properties of the product, declared once and enforced across every surface in §5. The Data Product Team operates the product; Governance defines the policies the product operates within. See [Part 1 §8](#) for the role split.

Lineage. Column-level lineage traces from source systems, through Nilus ingestion, through Vulcan transformations, into the materialized artifact, and onward to every consumption surface. This is lineage by construction, not lineage reconstructed from logs after the fact. Auditors, agents, and compliance reviewers all consume the same lineage graph.

Observability. Freshness, completeness, quality, run history, and cost are emitted as first-class signals from every stage. Pipelines, dashboards, pagers, and AI agents all gate on the same signals. A Data Product that is stale, failed, or degraded announces itself rather than being discovered by a consumer.



Without these horizontals, DataOS would be a collection of stage-specific capabilities. With them, it is a single coherent platform: what's classified in Discovery is still classified in Consumption, what's owned in Production is owned end-to-end, and what's observed anywhere can be observed everywhere.

PART 2 · §7

Above the Data Products: the Semantic Layer

The guide to this point treats Data Products as the atomic unit: each owns its semantic model, each serves its consumers, each is sovereign over its own data. That is correct, and it is the right starting point. But it leaves a question unanswered: what sits *above* the collection of Data Products, and how does cross-product reasoning become possible?

Data Products do not know about each other. A marketing campaigns product has no built-in knowledge that campaigns influence revenue. The revenue ledger product has no built-in knowledge that bonus disbursement depends on it. Each carries its own bounded context (its semantic model); none carries the organization's view of how its products relate to each other. Semantic information is siloed at the product level *by design*, because that siloing is what makes each product independently deployable, independently owned, and independently versioned.

But organizational reasoning operates *across* products. When a CMO asks "*did our Q3 campaign drive revenue*," that question exists inside no single Data Product. Answering it requires a chain of reasoning that spans marketing data, revenue data, and an organizational understanding that campaigns *influence* revenue. That chain has to live somewhere.

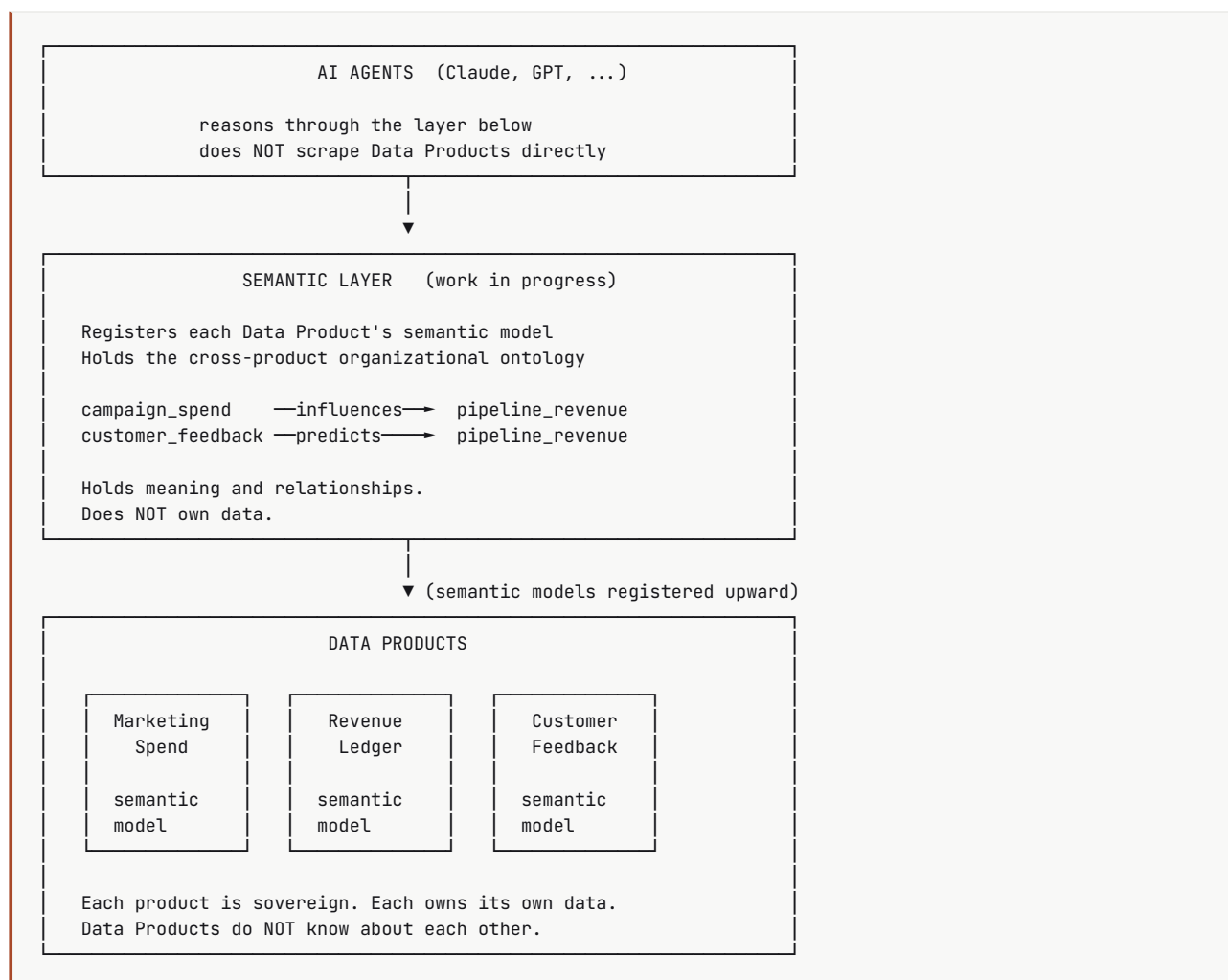
What the Semantic Layer is

The semantic layer is the architectural tier above the Data Products. It is the single layer where:

- Each Data Product's semantic model is registered and exposed.
- Cross-product relationships are made explicit: the organizational ontology records typed connections between concepts across products (`campaign_spend` *influences* `pipeline_revenue` ; `customer_feedback` *predicts* `pipeline_revenue`). These relationships are machine-readable.
- Audience- and context-aware framing becomes possible: the same query can surface differently for a CMO, an engineer, or an agent, because the layer encodes roles and personas alongside the ontology.

This layer goes by different names in different communities: *semantic backend*, *ontology layer*, *context graph*, *knowledge graph*. The naming varies; the function is the same. What matters is what the layer *does*, not what it is called.

The semantic layer does not own data. Data Products do. The layer holds the *meaning* of data and the *relationships* between Data Products. The Data Products remain sovereign; the layer is their interpretive context. This boundary is the single most important thing to get right: conflating the two (treating the layer as a data store, or treating it as part of any individual product) breaks the architecture.



Why this matters for AI

A Data Product can answer questions that live inside its bounded context. A semantic layer is what lets AI agents answer questions that *span* Data Products without hallucinating connections the data did not actually contain. The LLM already handles personalization and generation; what it cannot infer on its own is what your organization's data *means*, and how its concepts relate. The semantic layer is the mechanism by which that meaning becomes machine-readable.

Reasoning across products is only *safe* when the cross-product relationships are explicit and typed. Otherwise, an agent asked "*did the Q3 campaign drive revenue*" has to guess at how to join tables, which definitions apply, and which relationships are causal vs. coincidental. An agent operating over a semantic layer reads those relationships directly, rather than inferring them from schema and naming conventions.

Where this sits today, and what is coming

In DataOS today, each Data Product produces a fully-formed semantic model as part of Vulcan's build output (§4). That semantic model is what every consumption surface (§5) resolves through, and it is what makes train/serve skew eliminable *within* a product.

The cross-product semantic layer, the layer described in this section, is under active development. When it lands, it will make two new capabilities first-class in DataOS:

- **Cross-Data-Product AI reasoning.** Agents reason across the collection of Data Products using explicit, typed relationships, rather than inferring joins from column names. The organizational ontology supplies the meaning; the Data Products supply the governed data; the agent combines them with provenance preserved.
- **Cross-Data-Product data fusion.** Composition across products becomes a first-class operation, grounded in the ontology rather than ad-hoc SQL. Results carry their reasoning trace: *which* products contributed, *which* relationships were traversed, *which* definitions applied.

Both rest on the same foundation: the Data Products the rest of this guide describes. The discipline above does not substitute for the discipline below. An organization that builds its Data Products correctly (one owner, one contract, one engine, one semantic model, per Part 1) will have the atoms the semantic layer needs. An organization that treats its Data Products as pass-throughs will have nothing for the layer to integrate, and cross-product reasoning will collapse into the same hallucination problem AI agents face over raw warehouses today.

For the fuller external argument on why this tier matters in the AI era and what makes a semantic layer AI-ready, see [What Makes Knowledge Graphs AI-Ready](#).

PART 2 • §8

Full-stack and Overlay Patterns

Two realistic adoption shapes, both first-class.

Full-stack

The organization adopts DataOS end-to-end. The DataOS Lakehouse is the engine. Discovery uses Metis, Workbench, and Nilus together (Nilus bringing data into the Lakehouse). Vulcan builds Data Products. Consumption goes through the DataOS surfaces. Governance, lineage, and observability are DataOS-native across the board.

This pattern fits when:

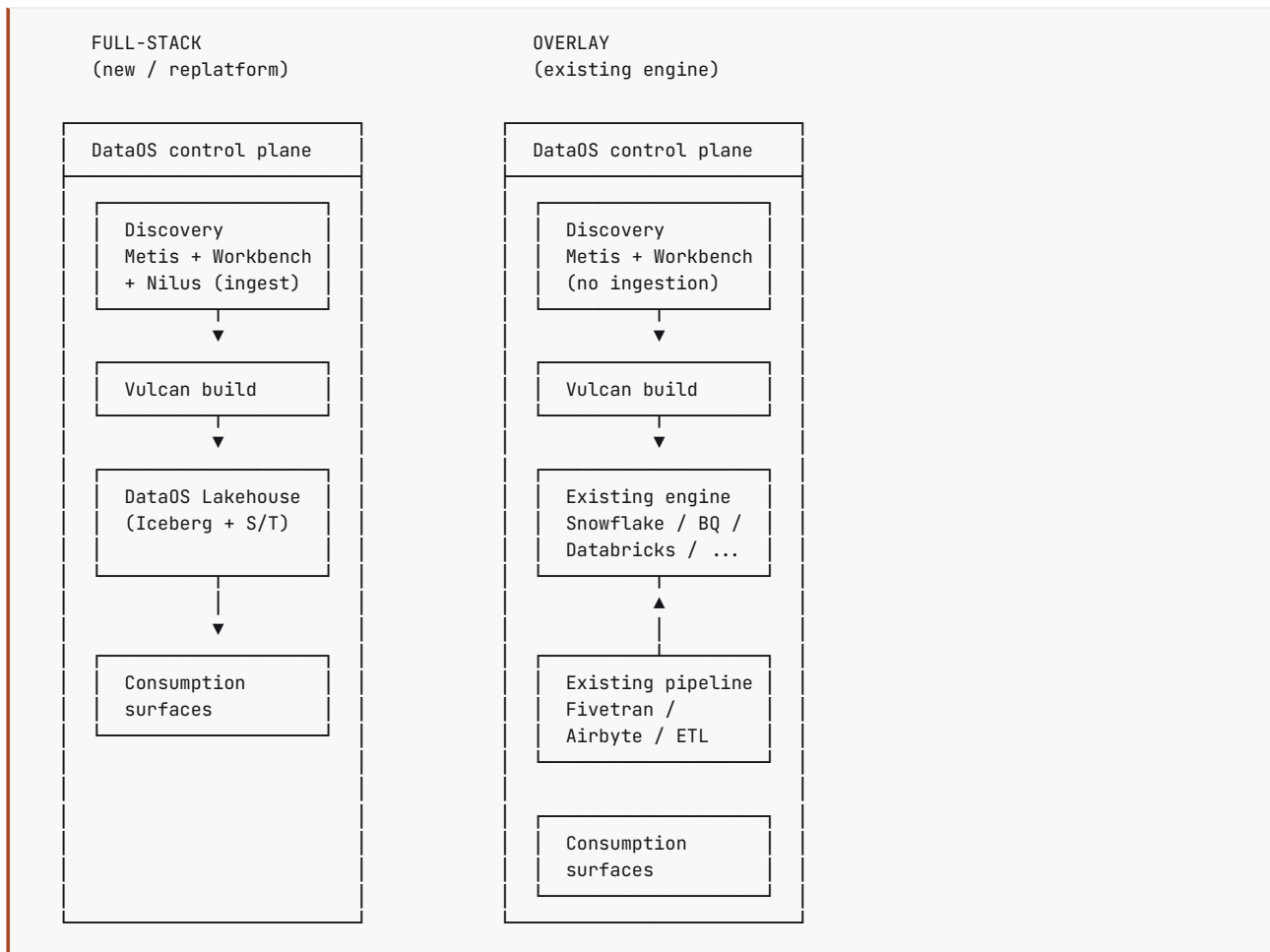
- The organization is building its data platform fresh, or undertaking a deliberate replatform.
- There is no pre-existing warehouse investment the team is committed to preserving.
- The team values a coherent single-vendor stack with minimal integration surface.

Overlay

The organization already has a governed engine (most often Snowflake, BigQuery, or Databricks) populated by existing pipelines (Fivetran, Airbyte, internal ETL). DataOS is adopted as an overlay: the existing engine stays, data is already landed, Discovery focuses on Metis and Workbench against what's already there (Nilus is typically not needed), Vulcan builds Data Products inside the existing engine, and consumers access through the DataOS surfaces.

This pattern fits when:

- The organization has a significant investment in an existing warehouse and its pipelines.
- The pain point is not ingestion or storage, but the discipline and consumability *above* the engine.
- The team wants Data Product discipline without a migration.



The two patterns are not a hierarchy. An organization can start with overlay for one set of products and full-stack for another, or migrate from overlay to full-stack over time, or stay in overlay permanently. What matters for the Data Products discipline is not which pattern is in use, but that the principles in Part 1 are upheld on whichever pattern is chosen: one owned artifact per product, one engine per product, one contract per product, a named owner, published quality, and versioning that respects consumers.

The engine is the choice. The discipline does not change.

